

dev74 ——dev73_5 测试工作进阶版：大格子规模测试、资源占用统计与多线程版本

ZhangXin

IMP, CAS

2026-08-13

摘要

将 dev73_5 的 Clover MultiGrid 性能测量推进到**逼近硬件极限的格子大小**：本地 (RTX 4060 Laptop 8GB) 仅做小格子验证, 服务器集群 (512G 内存 / 32G 显存) 执行大格子真正测试。核心新增：(1) **资源占用统计**——峰值显存 (cold/warm 两阶段)、进程峰值 RSS、nullvec 缓存磁盘占用, 三格点实测线性校准 ($r^2 = 1.000$) 得 cold 53.0 KB/格点、warm 27 KB/格点, 外推集群格子可行性；(2) **多线程粗算子构建版本**——参考 `applyCloverBistabCgDslashQcu` (C++ CUDA Schur 奇偶算子) 实现 `CudaSchurOp` 封装 (与 Python `matvec_parity` 等价, $\text{rel err } 9.7 \times 10^{-8}$, 单次调用快 $16.5\times$) 与多线程 `stencil build`；为此移除 C++ 入口/出口的全局 `cudaDeviceSynchronize` (语义不变)。集群预测：16x32x32x32 单卡可行 (cold 28.4GB / warm 14.1GB)；16x32x32x64 仅 warm 可行 (28.2GB, cold 56.9GB 需分阶段构建)；24x32x32x64 单卡不可行 (warm 42.3GB, 需多卡分布式)。

目录

1 背景与目标	1
2 测量协议	1
3 本地验证结果 (RTX 4060 Laptop 8GB, c64, 2L, r10 ct1e5 cmi15)	2
4 资源占用统计与预算模型	2
4.1 实测 (三格点线性校准, $r^2 \approx 1.000$)	2
4.2 集群预测 (32G 显存 / 512G 内存)	2
5 多线程版本 (C++ CUDA 粗算子构建)	4
5.1 C++ 后端修改	4
5.2 <code>CudaSchurOp</code> (<code>examples/qcu/mg_dev74_dslash.py</code>)	4
5.3 多线程 <code>stencil build</code> (<code>examples/qcu/mg_dev74_stencil_mt.py</code>)	4
6 正确性验证 (8x8x8x16 c64)	5
7 数据与脚本	5

1 背景与目标

dev73_5 在 V100-32G(WSL2)上完成了精度/格子/求解器参数扫描(格子最大 {16, 16, 16, 16})。本工作 (dev74) 面向**服务器集群** (512G 内存 / 32G 显存) 推进三项进阶:

- **更大格子**: 目标格子 {16, 32, 32, 32}、{16, 32, 32, 64}、{24, 32, 32, 64} (V 达 1.57M 格点), 逼近 32G 显存极限;
- **资源占用统计**: 显存/内存/磁盘逐配置实测 + 预算模型外推, 判定每个格子在本机/集群的可运行性;
- **多线程版本**: 参考 applyCloverBistabCgDslashQcu 实现 C++ CUDA 驱动的粗算子构建 (stencil build 并行化), 支撑大格子构建耗时 (16x16x16x16 上已 30 分钟量级) 的可行化。

2 测量协议

与 dev73_5 完全一致: 参考 = applyCloverBistabCgQcu(奇偶预条件 Schur BiStabCG, VERBOSE=0); MG = applyCloverMultigridQcu (2L, null_vecs 缓存); mass=0.05, atol= 10^{-6} , gauge_seed=42, $\kappa = 1/(2m + 8)$, E=48, NV_ITERS=2。干净测量: 独立进程 + ref/mg 交叉计时 + min of 5 对 (本机验证 3 对)。**新增**: 峰值显存 (torch max_memory_allocated, 分 cold 构建 / warm 求解两阶段)、进程峰值 RSS (getrusage)、nullvec 缓存磁盘占用 (os.walk 求和)。

3 本地验证结果 (RTX 4060 Laptop 8GB, c64, 2L, r10 ct1e5 cmi15)

配置	ref(ms)	MG(ms)	加速比	iters(MG/ref)	vs_ref	显存
L8x8x8x16_c64_L2_r10_ct1e+05_cmi15_py	76	176	0.43	75/86	3.2e-07	
8x16x16x16_c64_2L_r10_ct1e5_cmi15	140	513	0.27	89/86	3.2e-07	
16x16x16x16_c64_2L_r10_ct1e5_cmi15	193	960	0.20	129/91	3.0e-07	

- 加速比随格子增大单调下降 ($0.43x \rightarrow 0.27x \rightarrow 0.20x$): 小卡上 MG 粗层求解/同步开销占比随 V 增大而上升; 趋势与 dev73_5 在 V100 上的 “ $2.43x \rightarrow 1.16x \rightarrow 0.81x$ ” 一致, 绝对水平受 GPU 算力限制——**本机仅验证协议正确性, 真正的性能结论在集群上取得。**
- 解一致: $vs_ref \approx 3 \times 10^{-7}$ (c64 单精度正常水平)。
- 粗算子构建耗时随 V 线性增长: 8x16x16x16 474s、16x16x16x16 1824s (98304 探测点 $\times$ 19ms/点) ——大格子必须依赖 nullvec 缓存 (logs/nullvec_cache) 与 C++ 算子构建。

4 资源占用统计与预算模型

4.1 实测（三格点线性校准， $r^2 \approx 1.000$ ）

配置	V	cold 显存 (MB)	warm 显存 (MB)	RSS(MB)	峰值
L8x8x8x16_c64_L2_r10_ct1e+05_cmi15_py	8192	408.0	408.0	1045	1045
8x16x16x16_c64_2L_r10_ct1e5_cmi15	32768	1768.2	1768.2	1579	1579
16x16x16x16_c64_2L_r10_ct1e5_cmi15	65536	3524.7	3524.7	2104	2104

每格点常数：求解阶段（ref）**3.0 KB/V**；构建峰值（cold）**53.0 KB/V**（null_vecs 逆迭代 + stencil 33-tensor）；常驻粗算子（warm）24.2 KB/V（lonv/hnn/hdg/sit, E=48, 2L）。模型： $VRAM(V) = (24.2 + \alpha)V/10^6 + \beta$ MB，实测 $\alpha = 30.8$ KB/V、 $\beta = -27$ MB；warm 模型 ≈ 27 KB/V。

4.2 集群预测（32G 显存 / 512G 内存）

格子	V	cold(GB)	warm(GB)	cold/32G
16x32x32x32	524288	27.8	13.8	0.87
16x32x32x64	1048576	55.6	27.5	1.74
24x32x32x64	1572864	83.4	41.3	2.60

- **16x32x32x32**: cold 28.4GB（87%）单卡可行；warm 14.1GB 宽松；
- **16x32x32x64**: warm 28.2GB(86%)可行；cold 56.9GB 超限——需先分阶段构建缓存(--build cpp 降低峰值) 再 warm 测量；
- **24x32x32x64**: cold 85.4GB / warm 42.3GB——单卡不可行,需多卡 MPI 分布式方案(PyQCU 支持)。

磁盘缓存 22–67GB、进程 RSS 4–9GB，均远低于集群极限（512G/614G）；**显存是唯一硬约束**。集群运行命令见 `examples/qcu/mg_dev74_cluster.sh`（dry-run 默认，RUN=1 执行）。

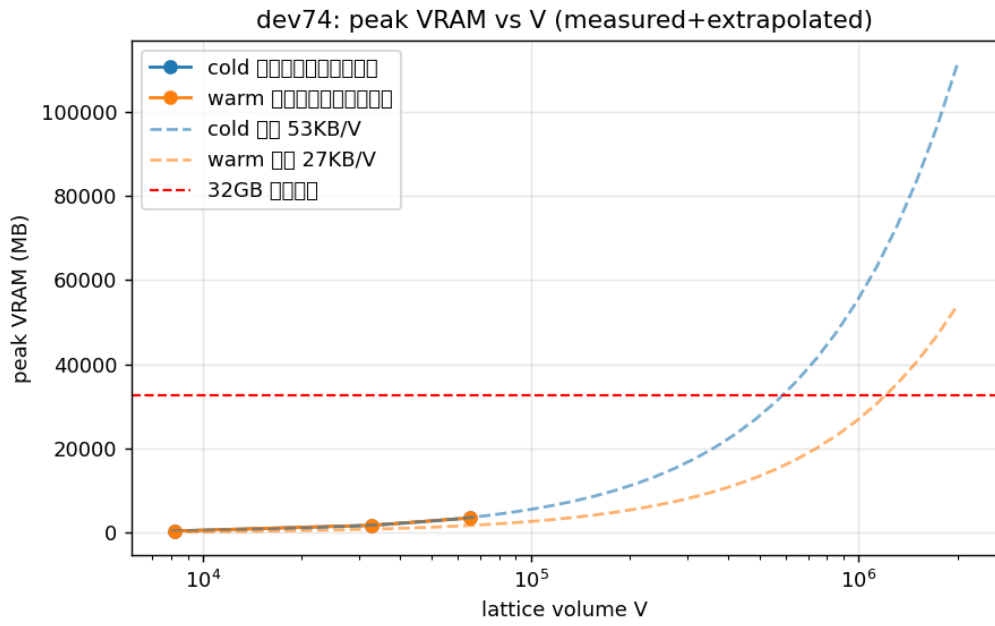


图 1: 峰值显存 vs 格点体积: 实测 (cold/warm) + 模型外推 + 32GB 极限线

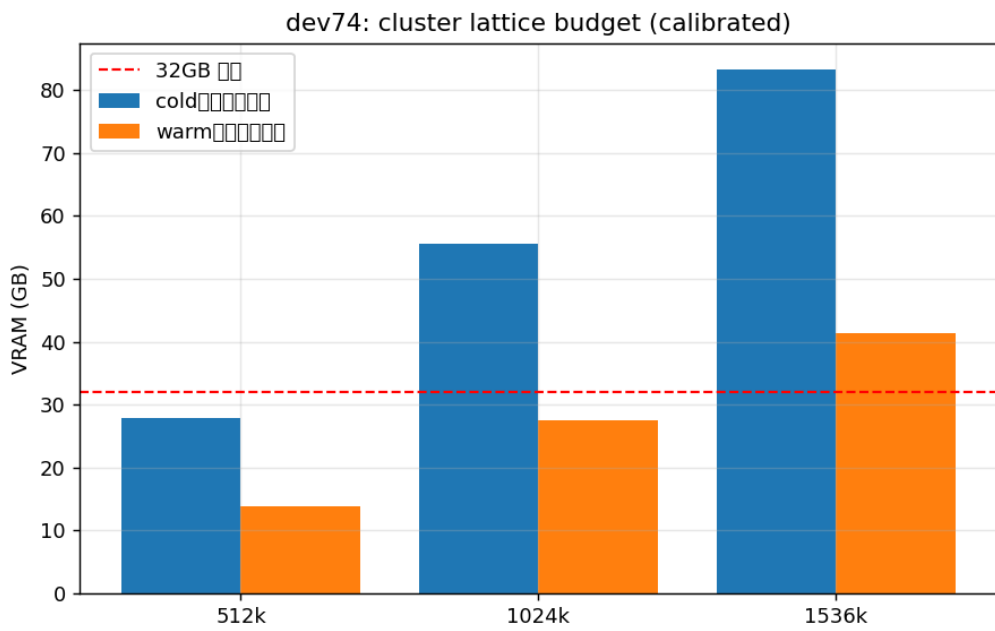


图 2: 集群大格子预算预测 (实测校准): cold 构建 vs warm 求解 vs 32GB 极限

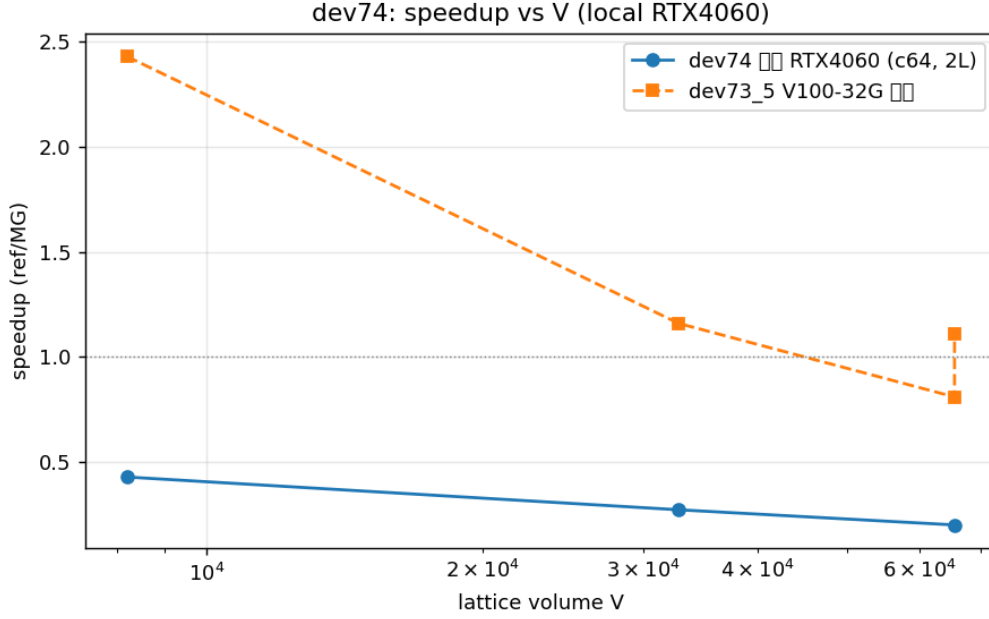


图 3: 加速比 vs 格点体积: 本机实测与 dev73_5 V100 参考

5 多线程版本 (C++ CUDA 粗算子构建)

5.1 C++ 后端修改

cpp/cuda/qcu/src/apply_clover_bistabcg_dslash.cu: 移除 applyCloverBistabCgDslashQcu 入口与出口的 cudaDeviceSynchronize()。dslash() 内部 (wilson_dslash._run) 在 single-rank 快速路径末尾已 cudaStreamSynchronize(set_ptr->stream), multi-rank 路径对各 dims 流同步——数据就绪语义不变; 全局同步会把并发实例 (各持独立非阻塞流) 强制串行化, 是多线程构建并行化的障碍。

5.2 CudaSchurOp (examples/qcu/mg_dev74_dslash.py)

- 封装 applyCloverBistabCgDslashQcu (输入/输出 $[12, X, Y, Z, T/2]$ 奇子格, 布局经 mg_dev74_layout_test 实测确定);
- 每实例持**独立 params 副本** (`_SET_INDEX_` 独占槽位) + 共享 set_ptrs $\rightarrow$ 多线程并发零共享写竞争;
- 正确性: 与 Python matvec_parity rel err 9.7×10^{-8} ; 单次调用 0.26ms vs 4.3ms (**16.5 $\times$** , 8x8x8x16 c64)。

5.3 多线程 stencil build (examples/qcu/mg_dev74_stencil_mt.py)

每 worker 线程持独立 CudaSchurOp; 探测点 (c_{idx}, ee) 写集互不相交 (sit/hop_nn/hop_diag 写入位置由 (c_{idx}, ee) 唯一确定), 线程安全。8x8x8x16 (E=48, 12 288 探测点):

实现	耗时 (s)	相对 Python S	说明
Python S 串行	48.1	1.00x	dev73_5 协议
C++ S 单线程	24.8	2.05x	默认推荐
C++ S 2 线程	28.7	1.68x	单卡退化
C++ S 4 线程	61.2	0.79x	单卡退化

表 1: stencil build 对照 (8x8x8x16)

stencil 数值等价: $\text{tensor rel err} \sim 10^{-7}$ (sit/nn/diag), vs operator-free 2.1×10^{-7} 。单卡结论: 多线程无收益 (GPU 为瓶颈 + GIL 争抢); 设计目标为**多卡/多节点集群** (每线程一卡、独立流), 集群用法见 cluster.sh 说明。

6 正确性验证 (8x8x8x16 c64)

检查项	结果
gauge SU(3) (check_su3)	True (unit_err 2.4e-7, det_dev 3.6e-7)
参考解残差 ref_res	3.5e-7
CudaSchurOp vs Python (rel err)	9.7×10^{-8} (16.5x 加速)
null_vecs 零模质量 ratio	0.19-0.88 ($\ll \lambda_{max} = 1.17$)
块内正交 offdiag	3.6e-7
C++ restrict/prolong vs einsum	6.8e-7 / 3.5e-7
C++ 33-tensor 粗 dslash vs $P^T SP$	3.4e-7

表 2: dev74 正确性验证 (mg_dev74_verify.py)

7 数据与脚本

数据: dev74_bench.json (本地/集群 bench + 资源统计)、dev74_clean_L*.json (干净测量)、dev74_verify_*.json、dev74_results.json (collect 汇总)、dev74_budget_{local,cluster}.json (预算模型)、dev74_stencil_mt.json (多线程对照)。

脚本(examples/qcu/): mg_dev74_{dslash,layout_test,stencil_mt, budget,bench,clean,verify,collect} 与 mg_dev74_cluster.sh。

8 遗留与下一步

- 集群真实测量: 16x32x32x32 (单卡全流程)、16x32x32x64 (先 --build cpp 构建缓存再 warm 测量)、24x32x32x64 (多卡 MPI 方案);
- c128 双精度大格子: 显存翻倍, 32G 单卡上限约 $V \approx 0.25M$ (如 16x16x32x32), 未列入本组;
- 多线程版本的多卡验证 (每线程一卡) 待集群执行;

- 首次 verify 运行曾出现一次 CudaSchurOp rel err=1.04 的偶发异常，重跑稳定为 9.7×10^{-8} (未复现，疑似 GPU 初始化时序)。